

AI in images and video processing

Thomas de Jaeger
thomas.de-jaeger@epita.fr

Advanced Topics: Facial & Emotion Recognition, Tracking

Session 6

Advanced Topics: Facial & Emotion Recognition, Tracking

Session 6

Objectives:

- Understand the key challenges in detecting faces in images
- Learn how face detection works, from classic to deep learning methods
- Explore how PCA and eigenfaces are used for classical face recognition
- Discover how modern systems use embeddings for face recognition
- Compare tools for face detection and recognition, and know their pros and cons
- Remove background and tracking

Face detection

Face detection: clearly distinguishes faces from other objects and establishes their boundaries

- It is the first step of any face analysis task
 - **Face tracking:** Detect and track detailed features in human faces that appear in images, videos or real-time camera feeds.
 - **Face analysis:** Age, emotion, gender
 - **Facial recognition:** Needs face detection to identify faces within an image and compared to stored faceprints
- **Output:** bounding boxes around detected faces

Face detection: methods

- **Haar Cascades:** Classic ML using handcrafted features. Runs on CPU. Implemented in OpenCV.
- **HOG + SVM** – Traditional approach using edge gradient features. Used in dlib. Runs efficiently on CPU. No deep learning.
- **CNN-based detectors (e.g., MTCNN, RetinaFace)** – Modern deep learning methods. Require GPU for best performance. Very accurate.
- **MediaPipe** – Recent lightweight model by Google. Combines ML + optimized graph inference. Real-time on CPU, cross-platform.

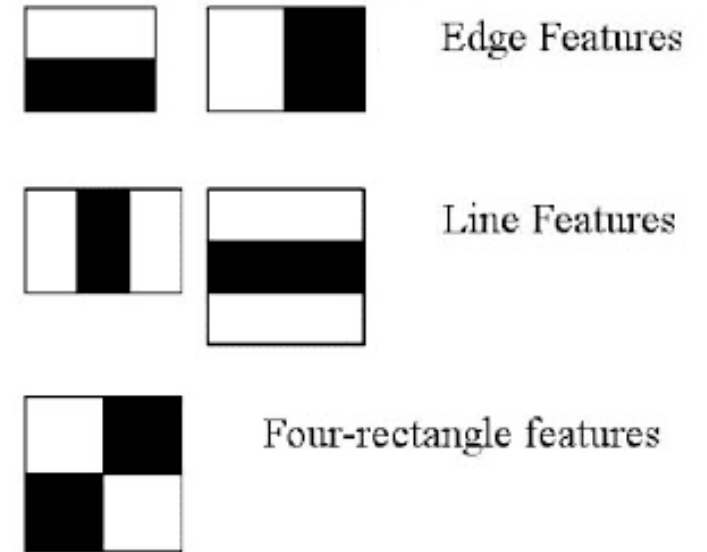
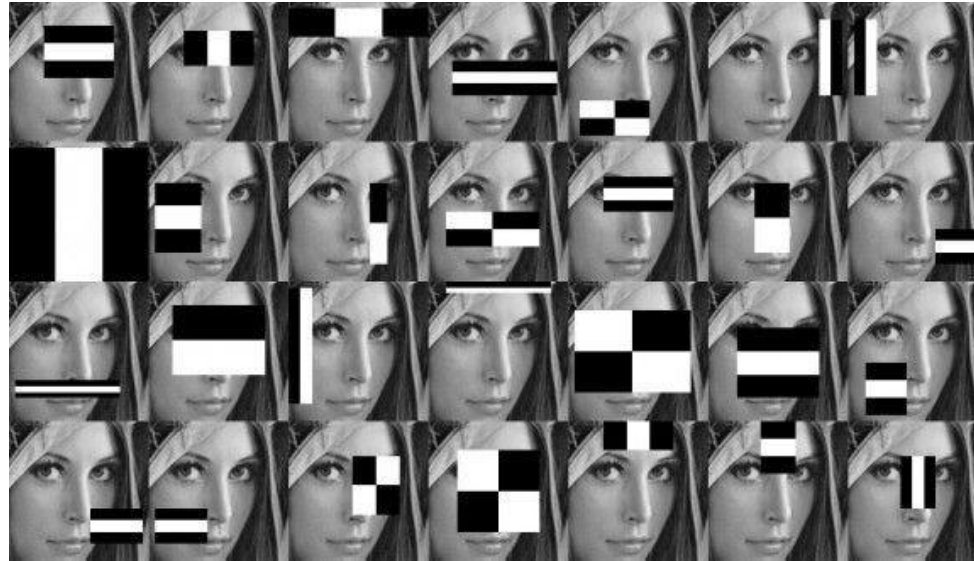
Face detection: Haar

Introduced in 2001, it's one of the earliest real-time detection methods. Works in four steps:

- Calculating Haar Features (e.g., edge, line, center-surround)
- Computes features rapidly using integral image representation
- Classifier is built using AdaBoost to combine weak learners
- Uses a cascade to reject non-face regions quickly and focus on likely area
- Pros:
 - Extremely fast on CPU
 - Suitable for embedded and real-time systems
- Cons:
 - Poor performance on rotated or occluded faces
 - Many false positives or missed detections

Face detection: Haar

- **Calculating Haar Features:** A **Haar-like feature** is a simple **rectangle pattern** (typically white and black areas) used to detect visual structures in an image.



- You just **sum the intensities** in the white and black regions.
- The difference of these sums becomes the feature's value.
- You slide (or "scan") it over the image at various scales and positions.

IMAGE MUST BE IN GRAY

Face detection: Haar

- **Integral image representation:** is a way to pre-compute cumulative sums of pixel intensities over a 2D image so you can quickly calculate the sum over any rectangular region.

A-----B
| |
C-----D

$$\text{Sum} = II(D) - II(B) - II(C) + II(A)$$

$$II(x, y) = \sum_{i=0}^x \sum_{j=0}^y I(i, j)$$

1.

3	1	4	1	5	9
2	6	5	3	5	8
9	7	9	3	2	3
8	4	6	2	6	4
3	3	8	3	2	7
9	5	0	2	8	8

2.

3	4	8	9	14	23
5	12	21	25	35	52
14	28	46	53	65	85
22	40	64	73	91	115
25	46	78	90	110	141
34	60	92	106	134	173

$$6 + 2 + 6 + 8 + 3 + 2 = 27$$

$$28 + 110 - 65 - 46 = 27$$

Face detection: Haar

Classifier ADABOOST: is an ensemble learning method that: combines many **weak classifiers** (e.g., simple Haar feature rules) into a **strong, accurate classifier**.

Start with all training samples equally weighted.

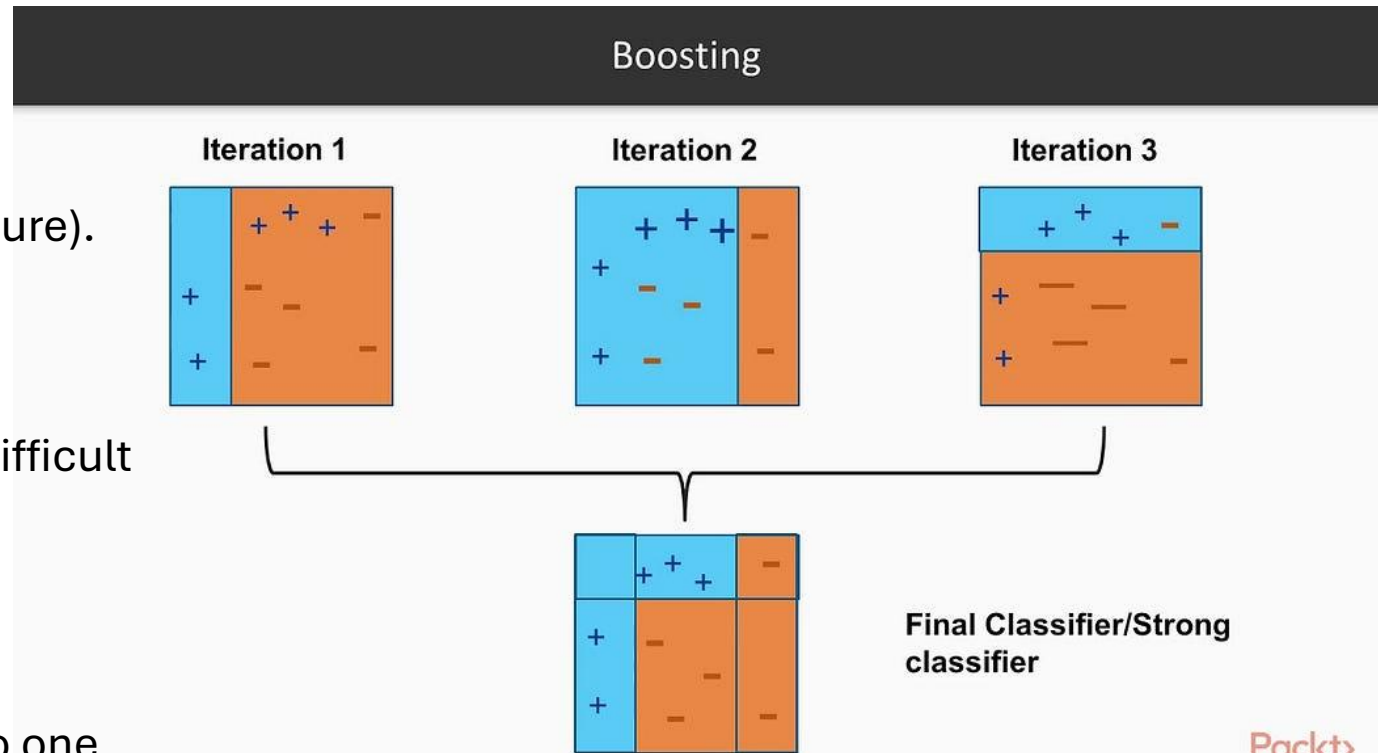
Train a weak classifier (e.g., a threshold on a Haar feature).

Increase weight on misclassified samples.

Train the next weak classifier — it tries harder on the difficult samples.

Repeat this for many rounds.

Combine all weak classifiers with learned weights into one strong decision function



Face detection: Haar

A cascade classifier: is a sequence of simple classifiers (stages), arranged in increasing complexity.

At each stage, the system quickly decides whether a region might contain a face.

- If not, the region is immediately rejected.
- If yes, it moves on to the next, more accurate (and costly) test.

Why a Cascade?

- Most regions in an image do not contain faces.
- No need to apply expensive computation everywhere.
- Early rejection = speed boost.

Face detection: HOG+SVM

- **Used in the dlib library and face_recognition package**
- **Extracts edge directions (gradients) from small regions (cells)**
- **Combines multiple cells into blocks to normalize lighting**
- **Final descriptor → fed into a linear SVM classifier**

How it works:

- Convert image to grayscale
- Compute gradient magnitudes and angles
- Bin angles into histograms
- Slide window across image
- Classify each window using SVM

Pros:

- Simple and interpretable
- Works well on frontal faces

Cons:

- Not robust to extreme poses or lighting
- Slower than CNNs on large images

Face detection: CNN-Based Face Detectors

- **These are deep learning models trained end-to-end to detect faces in complex scenes.**
- **Examples include: MTCNN, RetinaFace, BlazeFace (Google), YOLO-face.**
- **How they work:**
 - Use Convolutional Neural Networks to automatically learn features from training data
 - Detect multiple faces and return bounding boxes and landmarks
- **Pros:**
 - Highly accurate
 - Robust to lighting, pose, and scale variations
- **Cons:**
 - Slower than classical methods on CPU
 - Requires more computational resources (best with GPU)

Face detection: MTCNN

The **Multi-Task Cascaded Convolutional Neural Network (MTCNN)** detects faces through a four-stage pipeline:

- **Stage 0 – Image Pyramid (Scale-Invariance)**

To detect faces at different sizes, an image pyramid is created by downscaling the input image to multiple resolutions. These scaled images are passed through the following stages to ensure scale-invariant detection.

- **Stage 1 – Proposal Network (P-Net)**

The first network, P-Net, is a fully convolutional network that scans each image in the pyramid to generate candidate face regions and associated bounding box regression vectors. These regression vectors refine the bounding boxes, and Non-Maximum Suppression (NMS) is applied to merge overlapping detections. While fast, P-Net produces many false positives.

- **Stage 2 – Refine Network (R-Net)**

The candidate windows from P-Net are passed to R-Net, a deeper CNN that further filters out false positives. It also performs bounding box refinement and applies another round of NMS to improve localization accuracy.

- **Stage 3 – Output Network (O-Net)**

In the final stage, O-Net refines the remaining bounding boxes with greater precision and outputs five facial landmarks (eyes, nose, and mouth corners). This enables not just face detection, but also face alignment.

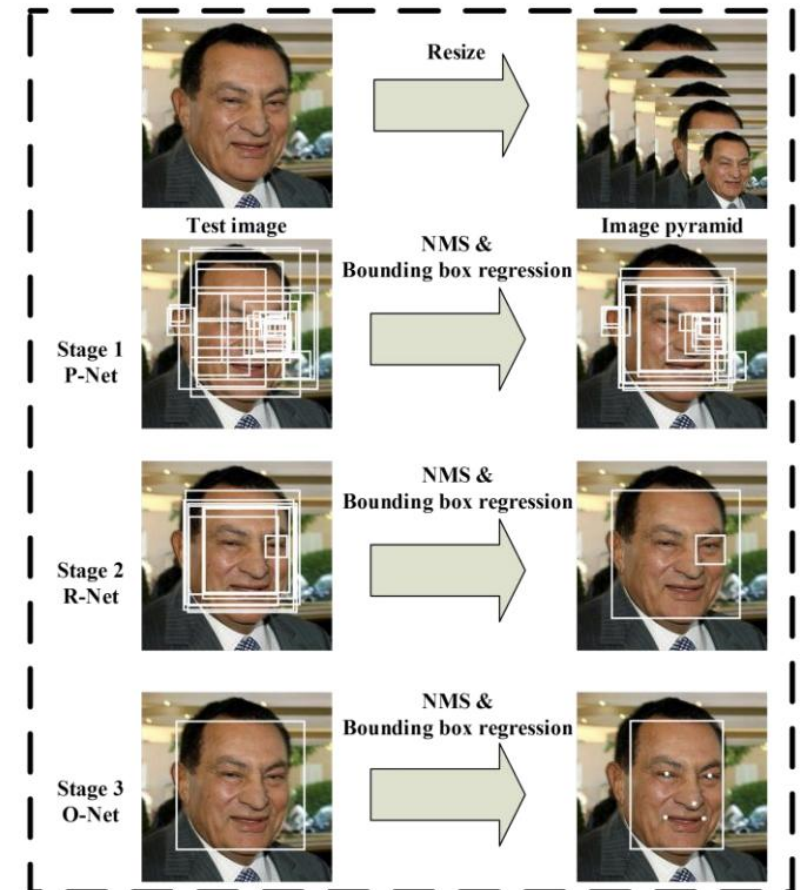


Fig. 1. Pipeline of our cascaded framework that includes three-stage multi-task deep convolutional networks. Firstly, candidate windows are produced through a fast Proposal Network (P-Net). After that, we refine these candidates in the next stage through a Refinement Network (R-Net). In the third stage, The Output Network (O-Net) produces final bounding box and facial landmarks position.

Face detection: MediaPipe

- **Developed by Google; real-time framework for lightweight face and hand tracking**
- **Optimized for mobile/web platforms; high performance on CPU**
- **Pros:**
 - Fastest among methods (real-time on CPU/mobile)
 - High-quality mesh estimation
 - Good robustness in practical environments
- **Cons:**
 - Less transparent (closed-source models)
 - Customization is limited
 - It's optimized for real-time webcam use, not high-density crowd scenes.

Face Recognition – Overview

Once a face is detected, the next step is to recognize or verify it.

- **Two major categories of face recognition systems:**
 - Classical methods: appearance-based, use PCA, LDA, etc.
 - Modern deep learning methods: extract embeddings and match them

Face recognition: Why is it hard?

- **Variation in appearance:** you don't have only 1 template...

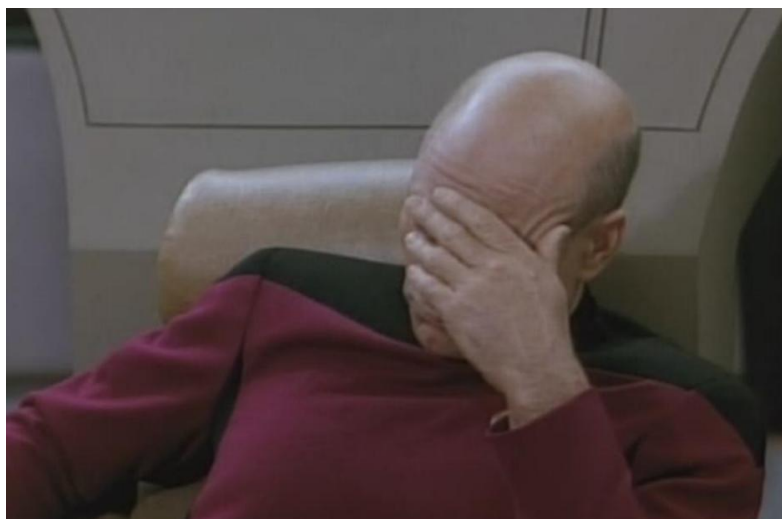


Face recognition: Why is it hard?

- **Lighting**



- **Occlusion**



- **Viewpoint**



Face recognition: simple approach

- Each face image is converted into a long vector representing its grayscale pixel intensities.
- Multiple images of each person, taken from different angles or with varying expressions, are stored in a model database during the training phase.
- During recognition, a grayscale vector is extracted from the unknown face image.
- This vector is compared to all vectors in the model database.
- The face in the database that is most similar (i.e., closest in appearance) to the unknown face is identified as the match.

Face regonition: simple approach

- **Problems :**

- Dimensionality of each face vector will be very large (250,000 for a 512X512 image!)
- Raw gray levels are sensitive to noise, and lighting conditions.

- **Solution:**

- Reduce dimensionality of face space by finding principal components (eigen vectors) to span the face space
- Only a few most significant eigen vectors can be used to represent a face, thus reducing the dimensionality

Classical Face Recognition – PCA & Eigenfaces

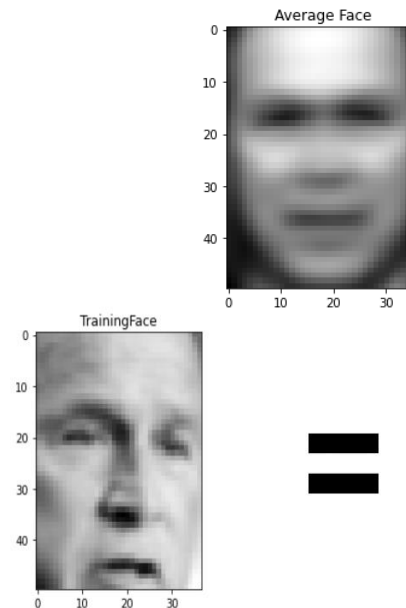
- Each grayscale image is flattened into a high-dimensional vector
- Use PCA to find the directions (eigenfaces) that capture the most variance (Center the data by subtracting the mean face and Compute covariance matrix and its eigenvectors)
- Each face is projected into the "face space" using the top eigenvectors
- During recognition, compare face vectors using Euclidean distance

Pros:

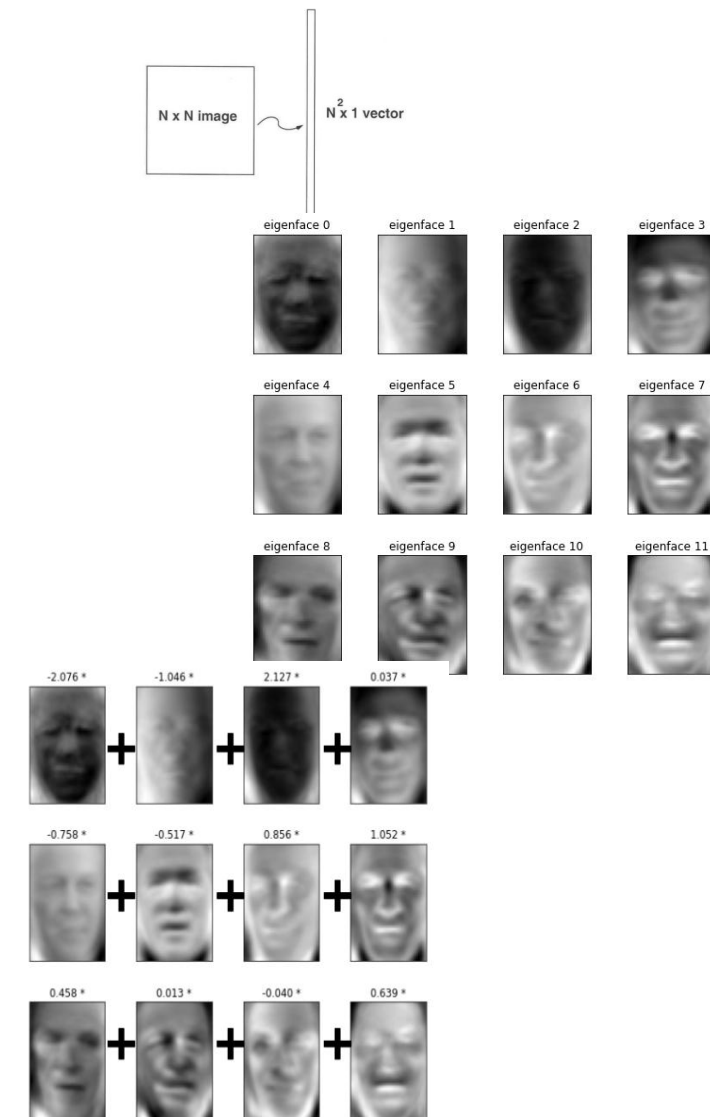
Simple and interpretable
Low computation requirement (Reduces 250,000-pixel image to ~100 dimensions)

Cons:

Sensitive to lighting, pose, occlusion
Requires aligned faces with fixed size



=



Deep Learning–Based Face Recognition with Embeddings

In modern face recognition, each face is converted into a short vector called an embedding. Faces that belong to the same person will have similar embeddings, and we can use this idea to recognize people.

- **Detect the face:** Use a deep model (like CNN or MediaPipe) to locate the face in the image.
- **Align the face:** Before recognition, it helps to align the face. That means rotating and scaling the face so the eyes and mouth are in the same place every time. This makes recognition more accurate.
 - **How it's done:** We use landmarks (like eyes, nose, mouth corners) to apply a small transformation. This centers and straightens the face.
- **Embedding:** Encode the face into an embedding. We pass the aligned face into a pretrained deep learning model (FaceNet, ArcFace). It outputs a small vector like 128 or 512 numbers. **This is the face's signature or identity vector.**
- **Compare embeddings:** to recognize a face, compare its embedding with others already stored. If the distance (usually cosine or Euclidean) is small, it's a match. If not, it's an unknown face.

Pro:

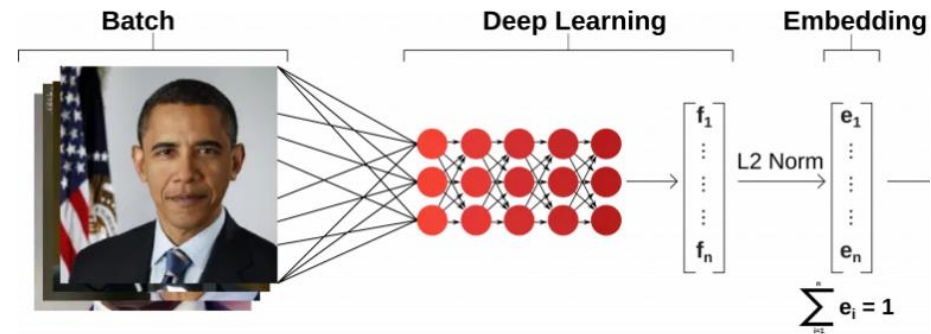
- Extremely accurate, even in real-world conditions (lighting, age, expressions)
- Easy to store, compare, and index for fast lookup, Scale well for large identity databases

Cons:

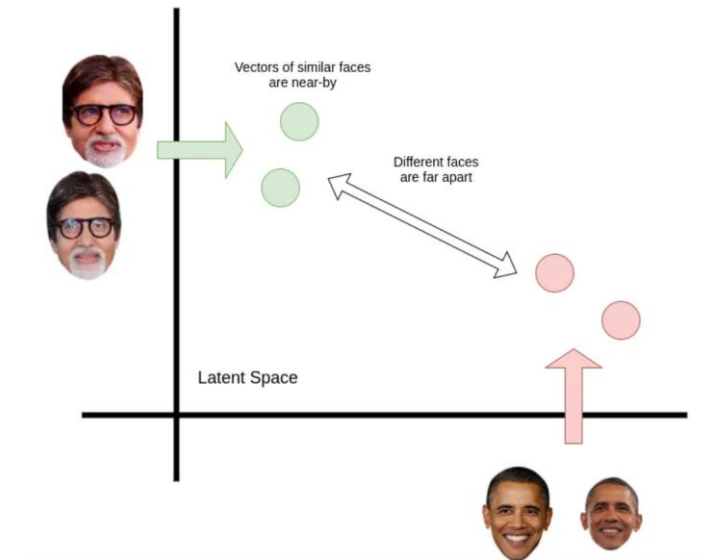
- Requires face alignment (landmarks or affine transform)
- Training a new model is GPU-intensive and requires lots of data

FaceNET

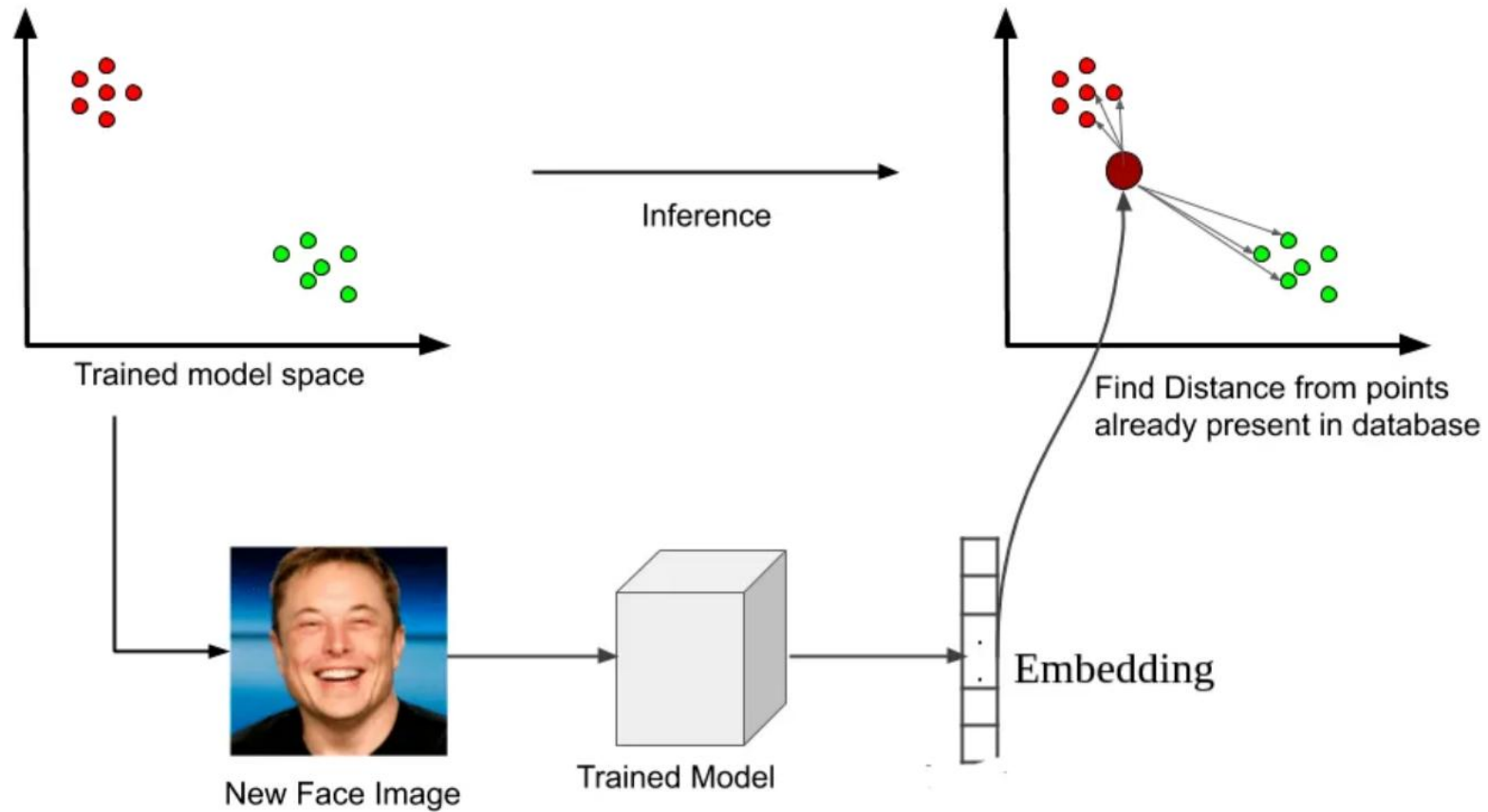
Takes input as a face image and returns a vector **of 128 numbers** which shows the most important features of a face, the vector is also known as **embedding**.



Embeddings are vectors and we can interpret vectors as points in the Cartesian coordinate system. That means we can plot an image of a face in the coordinate system using its embeddings



FaceNET



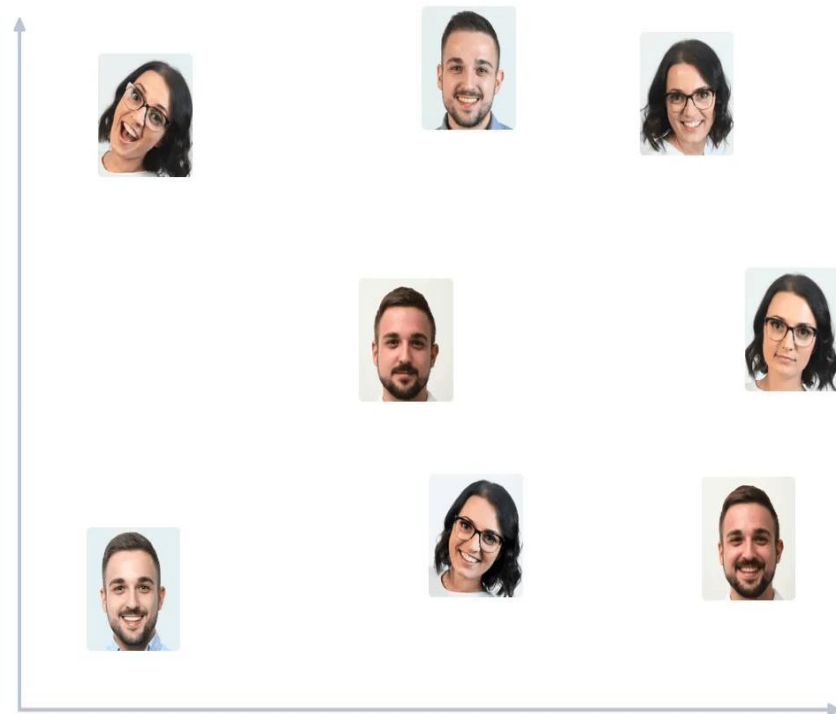
EASY!! But , how does FaceNet know what to extract from the image of a face and what do these numbers in an embedding vector even mean?

FaceNet

FaceNet learns in the following way:

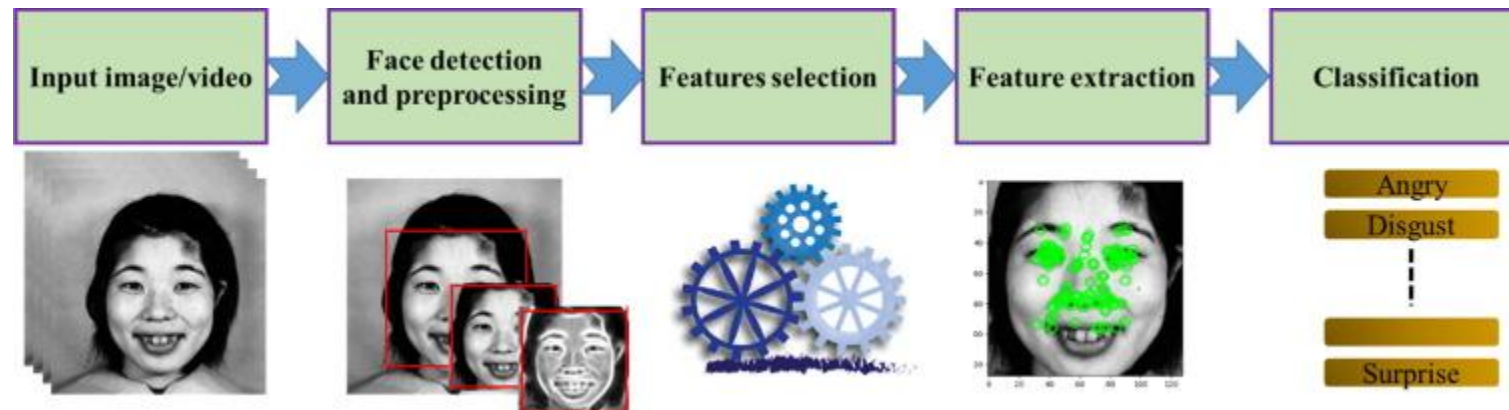
- Randomly selects an anchor image.
- Randomly selects an image of the same person as the anchor image (positive example).
- Randomly selects an image of a person different than the anchor image (negative example).
- Adjusts the FaceNet network parameters so that the positive example is closer to the anchor than the negative example

layer	size-in	size-out	kernel	param	FLPS
conv1	220×220×3	110×110×64	7×7×3, 2	9K	115M
pool1	110×110×64	55×55×64	3×3×64, 2	0	
rnorm1	55×55×64	55×55×64		0	
conv2a	55×55×64	55×55×64	1×1×64, 1	4K	13M
conv2	55×55×64	55×55×192	3×3×64, 1	111K	335M
rnorm2	55×55×192	55×55×192		0	
pool2	55×55×192	28×28×192	3×3×192, 2	0	
conv3a	28×28×192	28×28×192	1×1×192, 1	37K	29M
conv3	28×28×192	28×28×384	3×3×192, 1	664K	521M
pool3	28×28×384	14×14×384	3×3×384, 2	0	
conv4a	14×14×384	14×14×384	1×1×384, 1	148K	29M
conv4	14×14×384	14×14×256	3×3×384, 1	885K	173M
conv5a	14×14×256	14×14×256	1×1×256, 1	66K	13M
conv5	14×14×256	14×14×256	3×3×256, 1	590K	116M
conv6a	14×14×256	14×14×256	1×1×256, 1	66K	13M
conv6	14×14×256	14×14×256	3×3×256, 1	590K	116M
pool4	14×14×256	7×7×256	3×3×256, 2	0	
concat	7×7×256	7×7×256		0	
fc1	7×7×256	1×32×128	maxout p=2	103M	103M
fc2	1×32×128	1×32×128	maxout p=2	34M	34M
fc7128	1×32×128	1×1×128		524K	0.5M
L2	1×1×128	1×1×128		0	
total				140M	1.6B



Facial Expression Recognition (FER)

- 1.Face Detection:** locate the face(s) in an image or video frame.
- 2.Face Alignment** (optional): standardize scale and orientation.
- 3.Preprocessing:** resize to 48×48 or 224×224 , convert to grayscale or normalize.
- 4.Emotion Classification:** feed face into a model (usually a CNN) to predict one of 7–8 emotions (generally Angry, Disgust, Fear, Happy, Sad, Surprise, Neutral)



Background subtraction

Background subtraction is a technique to separate moving foreground objects from a static background in video streams.

Useful for:

- Person tracking
- Motion detection
- Surveillance

Works by learning a background model and subtracting current frame → areas with large differences are foreground.

- Each pixel maintains a history of recent values.
- These values are modeled as a mixture of Gaussians.
- The most stable distributions are assumed to be background.
- New values that do not match any Gaussian are marked as foreground.

Project presentation

Session 7